

HexOpt: Efficient and robust hexahedral mesh optimization using Rectified Hybrid Quadratic Jacobian and geometry-aware mapping[☆]

Hua Tong^a, Yongjie Jessica Zhang^{a,b},*

^a Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA

^b Department of Biomedical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA

ARTICLE INFO

Keywords:

Hexahedral mesh optimization
Jacobian
Mapping
Feature preservation
Augmented Lagrangian

ABSTRACT

We present *HexOpt*, a novel software package designed to optimize hexahedral mesh quality through a Rectified Hybrid Quadratic Jacobian energy functional and geometry-aware mapping. *HexOpt* accepts a triangular surface mesh and a volumetric hexahedral mesh as inputs. To overcome critical challenges such as element size-dependent gradient biases, prolonged optimization times, and erroneous boundary projection, we formulate a constrained optimization problem. The Rectified Hybrid Quadratic Jacobian energy is set as the objective function, while equality constraints enforce precise alignment between the quadrilateral boundary and the input geometry. These constraints fix vertices at geometric corners, restrict edge points to edges, and map face points to triangular faces. The optimization is solved via the Augmented Lagrangian method, with the Limited-Broyden-Fletcher-Goldfarb-Shanno method and the Armijo line search method driving the iterative process. The geometry-aware mapping employs normal-guided closest-point projection and mean value Tutte embedding to ensure geometric fidelity. *HexOpt* demonstrates robust performance across diverse 3D models and hexahedral meshes generated by various methods. It achieves consistent quality improvements without requiring manual intervention or parameter tuning compared with the initial state. Experimental results validate its computational efficiency and robustness in complex geometries, underscoring its utility as a reliable post-processing tool for hexahedral mesh optimization. We provide the *HexOpt* source code, input/output meshes (around 100 examples), and statistical data in the repository: <https://github.com/CMU-CBML/HexOpt>.

1. Introduction

Hexahedral (Hex) mesh generation plays an important role in solving partial differential equations in computer graphics, medical modeling, and engineering simulations [1]. Compared to tetrahedral meshes, hex meshes are sometimes preferred due to their higher accuracy and fewer element counts in bulk regions [2,3]. Despite these recognized benefits, automatic generation of high-quality and conforming hex meshes remains a significant challenge [4–6]. The generation of high-quality hex meshes typically involves (1) initial mesh generation with connectivity designed to fit the input geometry; (2) vertex relocation to optimize the mesh quality and geometry fitting [4]. The initial meshes that serve as input to quality improvement often contain poorly shaped and even inverted elements. On one hand, eventually all the inverted elements need to be eliminated, because even a single inverted, or non-convex element, brings challenge to simulation. On the other hand, unlike flexible quadrilateral (quad) or tetrahedral meshes, the rigid structure of hex elements complicates local adjustments [7].

Due to the aforementioned reasons, hex mesh optimization continues to be a dynamic and challenging field of research [8–13]. The core objectives of this process include optimizing the quality of poorly shaped elements and aligning the generated quad boundary with the input triangular boundary. Numerous methodologies have been developed to address these challenges. Among these, Laplacian smoothing stands out for its simplicity and computational efficiency, as it relocates vertices to the centroids of their connected neighbors [14]. However, while this method is cost-effective and straightforward, it risks inverting elements in concave regions, and it cannot fit quad boundary to the input triangular boundary precisely.

To overcome such shortcomings, optimization-based techniques have gained prominence. These strategies focus on iteratively refining the local mesh quality around nodes, often leveraging mathematical frameworks to ensure robustness [15–17]. Hybrid approaches that integrate Laplacian smoothing with optimization-based methods have also emerged, offering a balanced compromise between computational

[☆] This article is part of a Special issue entitled: 'SIAM IMR 2023/24' published in Computer-Aided Design.

* Correspondence to: Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA.

E-mail address: jessicaz@andrew.cmu.edu (Y.J. Zhang).

URL: <https://www.andrew.cmu.edu/user/jessicaz/> (Y.J. Zhang).

efficiency and stability [14,18]. Recent advancements have further extended these principles to non-manifold hex meshes, such as those found in micro-structured materials. For instance, a vertex classification system combined with pillowing, geometric flow, and targeted optimization is introduced [19], effectively addressing limitations in prior work related to element inversion and boundary alignment.

An untangling scheme performs single and fast local linear programming and traverses through each vertex until the quality cannot be improved any more [20]. Such method has local convergence proof, whereas it gets trapped in local minimum when the local solution space is empty, and better local solution can only be achieved by simultaneously moving multiple vertices. Therefore, an edge-cone rectification method that combines local quadratic programming with global reconciliation is employed to achieve good practical performance [21]. A similar Newton–Raphson-based method that maximizes the average scaled Jacobian is proposed [22], whereas this method does not consider fitting the geometry boundary.

Several other iterative techniques have been proposed to gradually shift vertices toward boundaries while avoiding local adjustments that might result in hex flipping [23–25]. These methods are straightforward but may sometimes struggle to maintain precise geometry, as each vertex will have a target projection point on the triangle boundary, but deriving the correct projection point is computationally expensive. To improve geometry fitting, a global deformation method has been shown to exhibit robust alignment of the resultant mesh with the input boundary through mapping. The fitting process is controlled using a Hausdorff distance threshold [26]. However, this method sometimes fails to preserve exact geometry in their experimental results, and the computation usually takes hours. A structure-aware geometric optimization method for hex meshes is proposed [27], aiming to improve mesh quality by optimizing the positions of singular lines and parameterization in the base complex structure. However, it cannot optimize hex meshes with tangled meshes and struggles with meshes featuring high-valence edges. A three-stage optimization process for all-hex meshes is also proposed [28], which is effective in producing high-quality, inversion-free meshes. However, this approach lacks the capability of projecting quad boundaries onto triangular counterparts.

In this paper, we present a new method with the software package *HexOpt* that begins with a manifold pure-3 simplicial complex equipped with annotated sharp features. Our approach introduces a new Rectified Hybrid Quadratic Jacobian (ReHQJ) objective function, which integrates two widely used algebraic metrics—the Jacobian and the scaled Jacobian. It is reformulated as quadratic measures through rectification and scaling. We analyze the scaled Jacobian function and mitigate its problematic behavior in degenerate regions. To evaluate optimization performance, we compare two strategies: steepest descent with a fixed learning rate (as implemented in the prior software package “HybridOctree_Hex” [29]) and Limited-Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) enhanced with Armijo line search. For surface vertices, we perform geometry-aware mapping by computing their projections onto the triangular surface using normal information and distances to individual triangular facets. For the most distant vertex from the triangular surface, we establish a bijective mapping between its one-ring closure of star (neighborhood) and its projection on the triangular mesh, ensuring correct projection direction. These projections are enforced as equality constraints for boundary vertices and managed through the Augmented Lagrangian (AL) method. Finally, we apply Laplacian smoothing to both boundary and interior vertices. Boundary smoothing improves boundary regularity, while interior smoothing together with Jacobian optimization propels internal vertices away from the quad boundary, accelerating convergence—particularly when the triangular geometry lies within the quad boundary. This enables inward deformation of the quad boundary to achieve tight conformal fitting during boundary optimization, which often occurs when the input triangular boundary has thin areas. In the experiments, we eliminate self-intersections and obtain optimal minimum scaled Jacobians across

all tested models, surpassing the current state-of-the-art results [21]. Additionally, our innovative approach excels in accurately preserving intricate curved and sharp features. To foster additional research and collaboration, we make available *HexOpt* source code, as well as a collection of around 100 optimized meshes and their input triangular and hex meshes, accessible at <https://github.com/CMU-CBML/HexOpt>.

The paper is organized as follows: Section 2 presents the algebraic shape quality metric, focusing on the (scaled) Jacobian for hex elements. It also explores the detailed algorithms of the AL objective function, the L-BFGS, and the Armijo line search, as well as the pipeline pseudo-code. Section 3 exhibits meshing examples, demonstrating that our proposed method creates valid meshes composed of high-quality hexes. Lastly, Section 4 concludes the paper and provides insight into potential future research directions.

2. Hex mesh optimization

The input is a watertight triangular mesh T annotated with sharp features and its corresponding hex mesh H . Here “watertight” means that each edge is shared by exactly two faces. The boundary of H is denoted as $bd(H)$. Although $bd(H)$ approximates the triangular mesh geometry, it lacks accurate fitting. To address this issue, we adjust H to H_f by warping vertices to ensure that $bd(H_f)$ fits T while maintaining a high minimum scaled Jacobian. Fig. 1 illustrates our mesh optimization process. The top left showcases the bunny geometry T . The bottom left shows the initial core mesh H in yellow and its boundary $bd(H)$ in blue. When the optimization begins, the gradient of the objective function measuring mesh quality and geometry fitting is calculated for each vertex in H_f , and the vertices are warped based on the approximated Hessian matrix. After some iterations, the vertices on $bd(H_f)$ first fit T , and the minimum scaled Jacobian increases; see the middle picture. Subsequently, we continue to optimize both geometry fitting and mesh quality until we can no longer improve the minimum scaled Jacobian without deviating vertices on $bd(H_f)$ from T . The final optimized mesh is shown in the bottom right.

2.1. Algebraic quality measures for hex elements

We adopt the scaled Jacobian to measure mesh quality [30]. Within each hex h , for every corner node x , three edge vectors are defined as $e_i = x_i - x$ ($i = 0, 1, 2$). The Jacobian matrix at x is constructed as $[e_0, e_1, e_2]$, and its Jacobian, $J(x)$, is the determinant of this matrix. We obtain the Scaled Jacobian, $SJ(x)$, when e_0, e_1 , and e_2 are normalized to $\frac{e_i}{\|e_i\|_2}$. For the scaled Jacobian $SJ(h)$ of a hex element h , we compute the scaled Jacobian at its the eight corners and the body center, and select minimum of these nine values. For the body center, e_i ($i = 0, 1, 2$) is calculated using three vectors formed by pairs of opposite face centers. The scaled Jacobian value range is $[-1, 1]$.

2.2. Rectified Hybrid Quadratic Jacobian

Since we want to maximize the minimum scaled Jacobian of H , the straightforward idea is to use the so-called Rectified Scaled Jacobian ($ReSJ$) as the objective function:

$$\max \sum_{h \in H} ReSJ(h, \theta), \quad (1)$$

where $\theta > 0$ is the threshold for the minimum scaled Jacobian of H ,

$$ReSJ(h, \theta) = \begin{cases} SJ(h), & \text{if } SJ(h) \leq \theta \\ \theta, & \text{if } SJ(h) > \theta. \end{cases} \quad (2)$$

Under this configuration, the optimization process is designed to target all hex elements with a scaled Jacobian below the threshold θ . The optimization terminates when the objective function reaches $N_h \theta$, where N_h denotes the total number of hexes in the mesh H . However, relying exclusively on the scaled Jacobian for optimization introduces three key challenges:

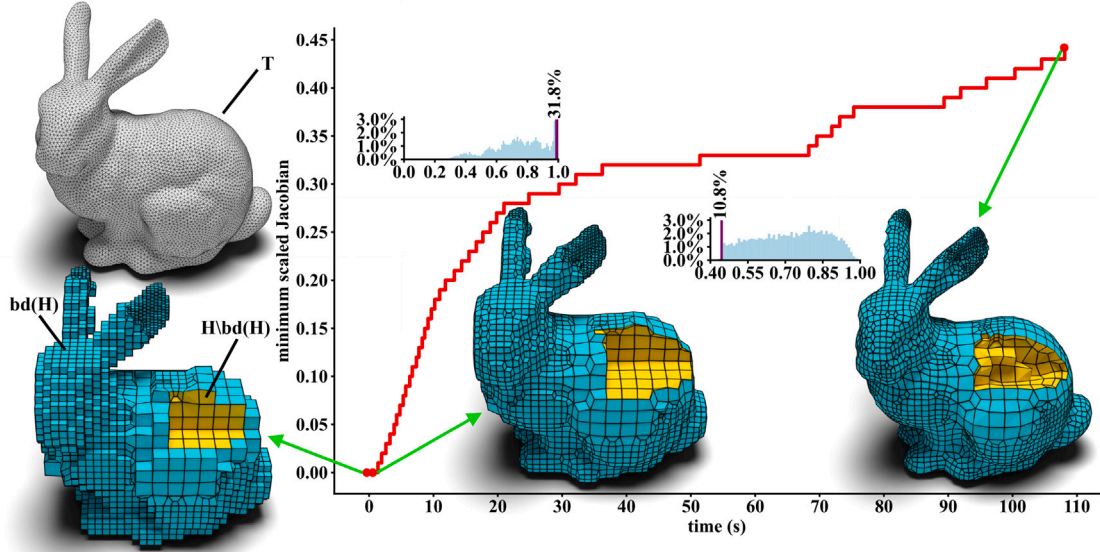


Fig. 1. Hex mesh optimization process. The triangle boundary T is shown in the upper left. The quad boundary $bd(H)$ is shown in blue. The hex core mesh $H \setminus bd(H)$ is shown in yellow. The vertices are warped to minimize the objective function in Eq. (8). The middle stage mesh H_f with $bd(H_f)$ closer to T with the minimum scaled Jacobian of 0.00. The minimum scaled Jacobian is increased by 0.01 every time with $bd(H_f)$ exactly fitting to T until we cannot improve anymore, and we export the final mesh at the bottom right with a minimum scale Jacobian of 0.40. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

1. The scaled Jacobian is non-differentiable at certain points and is inherently non-convex, even when the optimization is restricted to a single movable point (i.e., a 3-degree-of-freedom scenario). This issue is illustrated in the 3D top-view visualization in Fig. 2(a), where the points enclosed by red circles represent non-differentiable regions, while the areas marked by red squares indicate local minimum regions.
2. The scaled Jacobian is a dimensionless metric, and its derivative exhibits an inverse proportionality to hex edge length. Ideally, the derivative should scale proportionally (i.e., the objective function should be quadratic) with respect to element size to ensure optimization invariance under mesh scaling. Without this property, highly adaptive meshes suffer from gradient imbalance: fine-scale elements generate significantly larger gradients than coarse-scale elements, as demonstrated in [29].
3. When the quad boundary encloses the input triangular boundary, the former must contract inward to conform to the latter. This process frequently induces self-intersections, particularly in regions densely populated with elements between the quadrilateral and triangular boundaries. A straightforward solution would involve maintaining sufficient distance between internal and boundary vertices. However, since the scaled Jacobian does not account for absolute vertex distances, a new mixed functional is required to encourage internal vertices to repel one from another.

To address the first issue, we propose the improved Rectified Hybrid Jacobian (*ReHJ*). The idea is to leverage the convex and everywhere-differentiable landscape of the Jacobian (Fig. 2(b)) in the negative region to guide poorly-shaped elements toward a positive quality state. As illustrated in Fig. 2(c), both the Jacobian and the scaled Jacobian are always either positive or negative. When the Jacobian is non-positive, *ReHJ* is the Jacobian value; when the Jacobian is positive, *ReHJ* is the scaled Jacobian value. We obtain

$$ReHJ(h, \Theta) = \begin{cases} J(h), & \text{if } J(h) \leq 0 \\ SJ(h), & \text{if } J(h) > 0, SJ(h) \leq \Theta \\ \Theta, & \text{if } J(h) > 0, SJ(h) > \Theta. \end{cases} \quad (3)$$

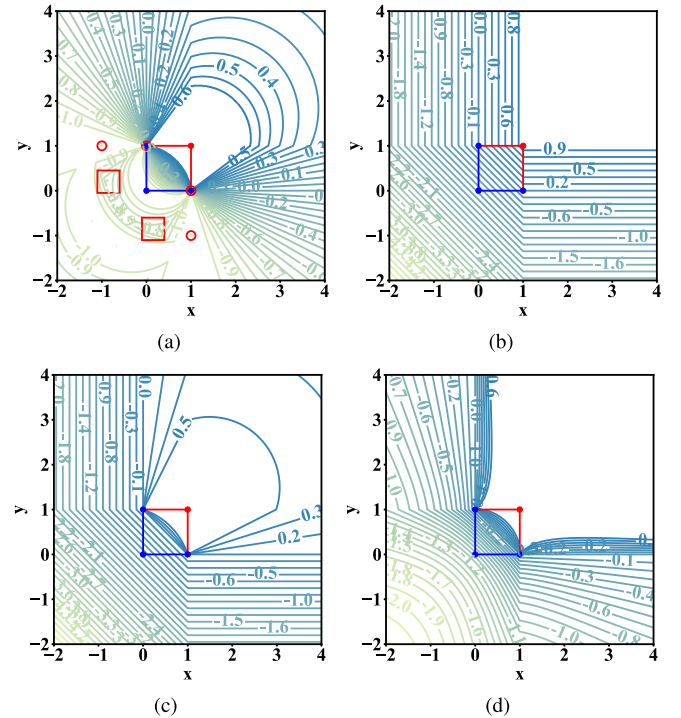


Fig. 2. J , $ReSJ$, $ReHJ$, and $ReHJQ$ plots of a hex element in the top view with six points (in blue) fixed at $(0, 0, 0)$, $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$, $(1, 0, 1)$, $(0, 1, 1)$, and two points (in red) with $z = 0, 1$ free to move within their z planes synchronously. $\Theta = 0.6$ is set. (a) Contour plot of $ReSJ$. Singular points are in red circles, and local minima are in red squares. (b, c) Contour plot of J and $ReHJ$, respectively. The functional landscape is piecewise linear in the negative Jacobian region. (d) Contour plot of $ReHJQ$ (final adopted objective function). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

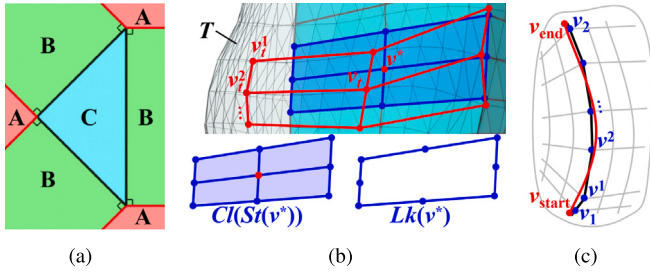


Fig. 3. Boundary constraints. (a) The planar domain of the triangle t is partitioned into three types of regions A (red), B (green), and C (blue). The closest projection point goes to a corner, an edge or the face if it lies in Region A, B or C, respectively. (b) The point v^* on $bd(H)$ is projected to v_i on the input triangular surface T . $\{v_1^i, v_2^i, \dots\}$ are a sequence of vertices on $Lk(v_i)$. The bottom two pictures show a closure of star surrounding v^* and its boundary forms the link of v^* denoted as $Lk(v^*)$. (c) For sharp edge vertices $\{v_{start}, v_{end}\}$, we identify their closest vertices $\{v_1, v_2\}$, compute the shortest path between them via Dijkstra's algorithm, and project intermediate vertices $\{v^1, v^2, \dots\}$ onto the pre-defined sharp feature curve. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

To address the second issue, recall that given three edge vectors e_0, e_1, e_2 at hex element h 's corner/center x , we have

$$J(x) = \det(e_0, e_1, e_2),$$

$$SJ(x) = \det\left(\frac{e_0}{\|e_0\|_2}, \frac{e_1}{\|e_1\|_2}, \frac{e_2}{\|e_2\|_2}\right), \quad (4)$$

where h 's average edge length is denoted as $\bar{e}$. We scale J and SJ to quadratic measures, namely QJ and QSJ , as follows:

$$QJ(x) = \frac{J(x)}{\bar{e}},$$

$$QSJ(x) = SJ(x)\bar{e}^2. \quad (5)$$

The updated objective function, called ReHQJ, is written as

$$ReHQJ(h, \Theta) = \begin{cases} QJ(h), & \text{if } J(h) \leq 0 \\ QSJ(h), & \text{if } J(h) > 0, SJ(h) \leq \Theta \\ \Theta, & \text{if } J(h) > 0, SJ(h) > \Theta. \end{cases} \quad (6)$$

To maintain consistent scaling of gradient contributions across different mesh elements, $\bar{e}$ is treated as a constant in Eq. (5), effectively normalizing the gradient of each vertex coordinate displacement relative to its corresponding element's edge length. This scaling strategy ensures that the gradient magnitude remains approximately proportional to the element size, preventing smaller elements from too large steps due to their inherently larger gradient magnitudes in scaled Jacobian. The landscape is illustrated in Fig. 2(d). Additionally, because the gradient of $ReHQJ$ is linearly proportional to edge length, scaling the mesh will yield identical optimization trajectories and final results.

To address the third issue, we introduce the Jacobian energy functional $J(h)$ instead of $ReHQJ(h, \Theta)$ as the objective for internal vertices. By leveraging the property that the Jacobian scales with the cube of the element edge length, this formulation encourages internal vertices to naturally repel away from boundary vertices during optimization. To preserve mesh validity, updates for affected elements are only permitted if they maintain the scaled Jacobian above a predefined threshold Θ .

However, optimizing $J(h)$ alone always propels vertices away from their neighbors, leading to severe tangling in the internal mesh. As a result, the scaled Jacobian can no longer be improved beyond a certain value. To mitigate this, we apply Laplacian smoothing to internal vertices. Unlike naive Laplacian smoothing, which updates a vertex v to $v_{new} = \lambda v + (1 - \lambda) \frac{\sum_{v_i \in V_{adj}} v_i}{N_{V_{adj}}}$ (a process prone to self-intersections and

Jacobian violations in concave regions, as noted in Figure 7 of [25]), our modified approach ensures stability. First, we identify the closest neighboring vertex $v_{close} = \arg \min_{v_i \in V_{adj}} \|v - v_i\|_2$. The update candidate v_{new} is then projected orthogonally to the vector $v_{close} - v$, yielding $v_{new2} = v_{new} - \frac{(v_{new} - v) \cdot (v_{close} - v)}{\|v_{close} - v\|_2^2} (v_{close} - v)$. This restricts movement to

the direction perpendicular to $v_{close} - v$, preventing reductions in the minimum edge length or excursions outside the feasible region. Similar to the Jacobian optimization, smoothing steps are only accepted if the minimum scaled Jacobian remains above Θ .

By combining Jacobian optimization with this modified Laplacian smoothing, *HexOpt* effectively handles scenarios where the boundary $bd(H)$ is gradually pushed inward into T , as illustrated by the bunny model in Fig. 1. The intermediate result H_f (middle model) demonstrates how the innermost layer of $bd(H)$ —the layer inside the pillowed layer—is proactively displaced inward. This preemptive adjustment avoids a computational bottleneck: if $bd(H)$ were naively moved inward, it could intersect with adjacent vertices, triggering repeated cycles of Jacobian optimization and boundary displacement (a computationally expensive feedback loop). The blue-yellow interface in H_f visually confirms the success of this strategy.

2.3. Geometry-aware mapping

Mesh quality optimization is constrained by the requirement that the boundary $bd(H)$ must be geometry-aware or conform precisely to the input triangular mesh T . During the initial optimization phase—prior to achieving an exact fit between the vertex set V and T —the boundary vertices are often positioned far from T . Experimental trials show that treating $bd(H)$ as a smooth boundary without sharp corners or edges during early optimization yields the most robust results. After convergence, $bd(H)$ forms an inscribed polyhedral approximation of T . At this stage, identifying sharp edges and vertices for geometric feature preservation becomes significantly easier, as the vertex displacements from warping are confined to localized regions. This localization minimizes global distortions and simplifies sharp feature constraints.

The natural idea is to adopt the edge-angle optimization method [26]. As shown in Fig. 3(a), for each vertex $v \in V$, we compute its closest projection v_i onto every triangle $t \in T$ by solving $v_i = \arg \min_{p \in t} \|p - v\|_2$. The projection v_i of v relative to triangle t falls into one of three types of geometric regions; see regions A, B and C in Fig. 3(a). First, each red corner region A is defined near one corner of t , formed by a fan-shaped area bounded by two rays perpendicular to the adjacent edges of t ; if v lies within a corner region, v_i coincides with the corresponding vertex of t . Second, each green edge region B adjacent to one edge of t is constructed by extruding rectangular areas bounded by two parallel rays perpendicular to the edge at its two endpoints; here, v_i becomes the orthogonal projection onto the corresponding edge. Third, a blue triangle region C is formed by the triangle t ; if v resides in this region, v_i corresponds to the orthogonal projection onto the plane of t .

To avoid the projection goes to the wrong side of the object especially for vertices in narrow regions, we incorporate normal directions at boundary vertices to ensure proper alignment of projection directions. Let N_v denote the set of right-hand (outward) normals at v , and let $\text{sgn}(v \rightarrow T) \in \{-1, 1\}$ indicate whether v lies inside (-1) or outside (1) of T . We compute the metric: $d_v = \sum_{n \in N_v} (1.1 \times \|v_i - v\|_2 + \text{sgn}(v \rightarrow T) (v_i - v) \cdot n)$. This rewards outward motion when v is inside ($\text{sgn} = -1$) and inward motion when v is outside ($\text{sgn} = 1$). The optimal projection is $\arg \min_{i \in T} d_v$.

Empirical observations reveal that vertices with the highest projection ambiguity correspond to the farthest vertices $v^* = \arg \max_{v \in bd(H)} \arg \min_{v_i \in T} \|v - v_i\|_2$. This is because for points far from T , multiple candidate regions on T might lie at similar distances from v , leading to ambiguity in establishing a unique correspondence. To resolve this, we compute a local bijective mapping for v^* instead of performing a

global surface mapping, which requires partitioning $bd(H)$ into genus-0 patches—a computationally expensive and unnecessary step, as experiments show over 99% of vertices on $bd(H)$ project correctly without such intervention. Here, “correctly” is defined operationally: in all meshes optimized in this work, we first perform an initial boundary projection by assigning each hex-mesh surface vertex v to its closest point on the triangular surface T . After running the optimization for a pre-defined time period (e.g., 100 s), we examine all vertices v and calculate the proportion of those that fail to lie on T due to inability to achieve a positive Jacobian relative to the total number of boundary vertices. As failed vertices are typically negligible in number, they are resolved through iterative local mapping: each time, only the farthest vertex from T is corrected, progressively eliminating these outliers without global partitioning. As shown in Fig. 3(b), our approach leverages the local neighborhood of v^* to guide its projection, assuming adjacent vertices (which form the boundary of a genus-0 patch) are more reliably projected. For efficiency and bijectivity, we adopt the Tutte’s embedding method, which suffices because the projection only needs to direct v^* approximately toward the correct region; subsequent Jacobian optimization and Laplacian smoothing will finetune its final position. The projection v_i of v^* is processed as follows:

1. Project each vertex $v^i \in Lk(v^*)$ onto the vertex set $V \subset T$ to obtain $v_i^j = \arg \min_{p \in V} \|v^j - p\|_2$, where $i = 1, 2, \dots$. Note that $Cl(St(v^*))$ is the closure of star surrounding v^* , and $Lk(v^*)$ is the link of v^* , which is also the boundary of $Cl(St(v^*))$.
2. Sequentially connect $\{v_i^1, v_i^2, \dots\}$ on T , preserving the adjacency order of $Lk(v^*)$. If adjacent projections v_i^j and v_i^{j+1} are not neighbors on T , we connect them via the shortest path on T ’s edges using the Dijkstra’s algorithm. The intermediate waypoint is excluded, and only the original v_i^j defines the closed loop.
3. Map $Cl(St(v^*))$ onto a regular polygon $P(v^*) \subset \mathbb{R}^2$, centered at the origin with edge count matching the number of vertices on $Lk(v^*)$.
4. Embed the projected loop $\{v_i^1, v_i^2, \dots\}$ (excluding the waypoint added using the Dijkstra’s algorithm) into $P(v^*)$ using Tutte’s embedding with mean value coordinates [38].
5. Compute v_i by transferring v^* ’s barycentric coordinates from $P(v^*)$ to the patch on T . This avoids the messy geodesic computations while leveraging the Dijkstra’s algorithm to handle gaps between non-adjacent v_i^j .

The bijectivity constraint ensures v_i lying near the correct region on T , even for imperfect projections. Combined with Laplacian smoothing on $bd(H)$, this efficiently fits $bd(H)$ onto T with regular quad patterns.

After initial fitting of $bd(H)$ onto T , we proceed to preserve sharp features. For each sharp corner vertex $v_{\text{corner}} \in T$, find its closest vertex on $bd(H)$ using $v = \arg \min_{u \in bd(H)} \|u - v_{\text{corner}}\|_2$ and project it onto v_{corner} . For a sharp edge $e_{\text{edge}} = \{v_{\text{start}}, v_{\text{end}}\} \subset T$ as shown in Fig. 3(c), we first locate its two endpoints on $bd(H)$ with $v_1 = \arg \min_{u \in bd(H)} \|u - v_{\text{start}}\|_2$ and $v_2 = \arg \min_{u \in bd(H)} \|u - v_{\text{end}}\|_2$. The shortest path $\{v^1, v^2, \dots\}$ between v_1 and v_2 on $bd(H)$ is computed using the Dijkstra’s algorithm. Here, v_1 is projected to v_{start} , v_2 is projected to v_{end} , and intermediate vertices $\{v^1, v^2, \dots\}$ along the path are projected onto e_{edge} . For each v^j , its cumulative path distance from v_1 is defined as $d(v^j) = \|v^1 - v_1\|_2 + \sum_{j=1}^{i-1} \|v^{j+1} - v^j\|_2$. Then we define a weight λ^i for v^i as $\lambda^i = \frac{d(v^j)}{d(v^n)}$, where $d(v^n)$ is the total path length from v_1 to v_2 . Finally, v^i is projected to the position $\lambda^i v_{\text{end}} + (1 - \lambda^i) v_{\text{start}}$. This formulation ensures preserving the sequence of intermediate vertices.

2.4. Augmented Lagrangian

The optimization problem is then formulated as:

$$\begin{aligned} \max \quad & \left(\sum_{h \in H} ReH QJ(h, \Theta) \right) \\ \text{subject to } & V^k = V_t^k, \end{aligned} \quad (7)$$

where $V^k \in bd(H_f)$ are vertices of the quad boundary in optimization iteration k . $V_t^k \in T$ denotes the target points to which V^k should project. $\Theta = 0$ is set initially, representing the desire for at least an all-positive-Jacobian mesh. Similar to Eq. (1), the optimization terminates when $\sum_{h \in H} ReSJ(h, \Theta) = N_h \Theta$ and all constraints are met. Subsequently, Θ is incremented by 0.01, the optimization of H_f is repeated with the new configuration, and the previous iteration solution is used as the “warm start”. This iterative process continues until the optimization problem becomes infeasible.

The constrained optimization problem (7) can be reformulated as an unconstrained minimization problem through the AL method:

$$\begin{aligned} \min L(H, \Theta, T) = \min \quad & \left(- \sum_{h \in H} ReH QJ(h, \Theta) + \right. \\ & \left. \sum_{v^k \in V^k} \left(\lambda_{v^k} (v^k - v_t^k) + \frac{\rho_k}{2} (v^k - v_t^k)^2 \right) \right), \end{aligned} \quad (8)$$

where λ_{v^k} is the Lagrangian multiplier, and ρ_k is the penalty parameter. Following each iteration, λ_{v^k} is updated:

$$\lambda_{v^k} = \lambda_{v^k} + \rho_k (v^k - v_t^k). \quad (9)$$

Each time the minimum scaled Jacobian reaches Θ (i.e., $\sum_{h \in H} ReSJ(h, \Theta) = N_h \Theta$), ρ_k is doubled to strengthen the boundary constraints.

2.5. L-BFGS and line search

An optimizer is essential for determining both the search direction and the step size for H to solve Eq. (8). While a simple approach would be to use gradient descent, given the assumption that $bd(H)$ closely approximates T , a quasi-Newton method with quadratic convergence [39] is employed to compute the search direction. In contrast, gradient descent employs the negative gradient $p_k = -\nabla_{x \in H} L$ directly to compute the search direction. Consequently, the computational time per iteration for gradient descent is shorter compared to quasi-Newton methods. However, due to its first-order convergence, gradient descent needs more iterations than quasi-Newton methods to achieve convergence.

Other quasi-Newton methods, including the Levenberg–Marquardt method [40], face a challenge: the calculation of the Hessian and the inversion of sparse matrices are computationally intensive, leading to a considerable rise in computational time. This level of computational demand is not acceptable for mesh optimization tasks involving millions or more variables. A more advantageous equilibrium between the convergence rate and computational complexity is attained through the L-BFGS algorithm. L-BFGS implicitly approximates the inverse Hessian H_k^{-1} for the k th iteration of the function $p_k = -H_k^{-1} \nabla_{x \in H} L$ by utilizing information from the preceding m optimization iterations. For estimating the Hessian, it retains data from prior iterations, specifically $s_k = V^{k+1} - V^k$ in optimization iteration k and $y_k = \nabla_{k+1} L - \nabla_k L$. Although the determination of the search direction using L-BFGS is approximately twice as slow as gradient descent in experimental settings, it requires about an order of magnitude fewer iterations to reach convergence.

After determining the search direction p_k , we need to select a proper step size a . Consider the Taylor expansion of L at V^k :

$$L(V^k + ap_k) = L(V^k) + ap_k^T \nabla L(V^k + tap_k). \quad (10)$$

The exact value of t remains unknown, thus we adopt the Armijo line search to numerically determine t . We introduce a small parameter $c_1 = 10^{-4} \in (0, 1)$ alongside $\nabla L(V^k)$ to relax the constraint on $L(V^k + ap_k) - L(V^k)$. Initially, we set $a = 1$ and accept this value if $L(V^k + ap_k) - L(V^k) \leq c_1 ap_k^T \nabla L(V^k)$. Should this condition not be met, the step size a is adjusted through backtracking by multiplying it by $\eta = 0.9$. It is important to note that, theoretically, an arbitrarily small step size could satisfy the aforementioned condition; however, adopting excessively small steps markedly increases the computational time. Consequently, we terminate the backtracking process once a drops below 10^{-8} .

Table 1

Hex mesh statistics for models optimized with *HexOpt* and competing methods. The Info column contains (from top to bottom): initial geometry name, hex mesh source from literature, number of vertices, number of hex elements, minimum scaled Jacobian before perturbation, and minimum scaled Jacobian after perturbation. Best values are bolded.

Info	Method	PostSJ	MaxDist	Time (s)	Info	Method	PostSJ	MaxDist	Time (s)
rkm012_1 [31]	<i>HexOpt</i> [32]	0.60 0.17	0.001756 0.001762	25/42 4442	mount2 [33]	<i>HexOpt</i> [32]	0.37 0.19	0.001147 0.001189	10/36 11,981
21,312 [28]		0.39	0.001787	103	7945 [28]		0.37	0.001191	289
18,751 [21]		0.30	0.001767	34	6208 [21]		0.33	0.001150	31
0.47 [23]		0.070	0.001789	6	0.14 [23]		0.019	0.001189	5
-1.0 [26]		0.41	0.001821	90	-1.0 [26]		0.33	0.001166	77
impeller [34]	<i>HexOpt</i> [32]	0.43 0.19	0.0007058 0.0007066	20/48 6966	mid2Fem [35]	<i>HexOpt</i> [32]	0.48 0.24	0.002783 0.002882	5/10 3362
15,248 [28]		0.38	0.0007178	95	1590 [28]		0.38	0.002793	104
11,174 [21]		0.25	0.0007090	32	908 [21]		0.36	0.002805	26
0.18 [23]		0.020	0.0007101	7	-0.15 [23]		0.020	0.002816	6
-1.0 [26]		0.28	0.0007123	88	-1.0 [26]		0.39	0.002827	76
bunny [36]	<i>HexOpt</i> [32]	0.44 0.24	0.0007040 0.0007138	8/18 4160	CAD4 [37]	<i>HexOpt</i> [32]	0.12 0.050	0 0	9/20 1892
3724 [28]		0.38	0.0007149	200	3721 [28]		0.17	0	108
2832 [21]		0.36	0.0007061	50	2704 [21]		0.18	0	31
-0.77 [23]		0.020	0.0007072	7	0.069 [23]		0.010	0	5
-1.0 [26]		0.39	0.0007081	73	-1.0 [26]		0.090	0	46
anc101 [23]	<i>HexOpt</i> [32]	0.33 0.16	0.002581 0.002586	69/198 13,244	isidore_horse [29]	<i>HexOpt</i> [32]	0.54 0.24	0.003946 0.003987	54/171 5468
154,675 [28]		0.27	0.002602	200	209,974 [28]		0.31	0.003991	121
135,982 [21]		0.17	0.002600	50	182,124 [21]		0.19	0.003996	79
0.017 [23]		0.056	0.002595	4	0.24 [23]		0.020	0.004002	5
-1.0 [26]		0.21	0.002646	86	-1.0 [26]		0.26	0.003959	86
i29o_bracket [29]	<i>HexOpt</i> [32]	0.23 0.16	0.006622 0.006689	145/260 10,118	grayloc [29]	<i>HexOpt</i> [32]	0.43 0.13	0.006481 0.006493	18/48 6160
403,755 [28]		0.10	0.006686	496	13,028 [28]		0.37	0.006499	20
358,808 [21]		0.050	0.006695	73	10,634 [21]		0.35	0.006495	17
0.13 [23]		0.010	0.006669	4	1.0 [23]		0.010	0.006511	7
-1.0 [26]		0.050	0.006689	82	-1.0 [26]		0.35	0.006509	78
bumpy_torus [29]	<i>HexOpt</i> [32]	0.56 0.18	0.005207 0.005283	22/41 4165	dancing_children [29]	<i>HexOpt</i> [32]	0.54 0.17	0.004612 0.004659	48/117 4046
16,601 [28]		0.39	0.005290	80	44,511 [28]		0.37	0.004666	128
13,705 [21]		0.30	0.005285	25	37,195 [21]		0.29	0.004683	41
1.0 [23]		0.070	0.005279	6	0.15 [23]		0.020	0.004652	6
-1.0 [26]		0.41	0.005248	80	-1.0 [26]		0.46	0.004687	94
armadillo [29]	<i>HexOpt</i> [32]	0.54 0.22	0.003691 0.003716	52/99 3475	woodenfish [29]	<i>HexOpt</i> [32]	0.55 0.23	0.004836 0.004880	32/50 4726
38,566 [28]		0.38	0.003752	140	12,625 [28]		0.39	0.004890	103
32,296 [21]		0.18	0.003779	85	10,029 [21]		0.28	0.004898	36
0.15 [23]		0.070	0.003719	5	0.15 [23]		0.020	0.004901	7
-0.99 [26]		0.29	0.003773	95	-0.99 [26]		0.42	0.004919	94
bimba [29]	<i>HexOpt</i> [32]	0.57 0.23	0.004435 0.004487	69/97 3760	greek_sculpture [29]	<i>HexOpt</i> [32]	0.57 0.15	0.003709 0.003781	39/74 4857
70,153 [28]		0.29	0.004499	308	46,106 [28]		0.37	0.003769	92
62,139 [21]		0.19	0.004479	79	39,609 [21]		0.29	0.003773	44
0.15 [23]		0.13	0.004495	5	0.15 [23]		0.064	0.003823	5
-1.0 [26]		0.26	0.004525	86	-1.0 [26]		0.46	0.003763	90

2.6. Complete pipeline pseudo-code

The pseudo-code in Algorithm 1 summarizes our proposed pipeline. The process of searching for V_i for V involves iterating over all corner points, edges, and faces, which introduces a significant computational overhead. Although confining the search to objects within a predefined searching box centered at each $v \in V$ typically yields similar v_i outcomes, global traversals are necessary to ensure the highest possible success of the optimization. The constants used in the pipeline are experimentally determined, and variables with subscripts less than zero are disregarded as they represent invalid or initialization states.

3. Results and discussion

We first evaluate our method on a range of input triangle geometries and hex meshes generated using various state-of-the-art algorithms archived in HexaLab [41] and from our group's previous work [29,33] on a PC equipped with a 3.6 GHz Intel i7-12700 CPU and 64 GB

of memory. These methods include PolyCube-based approaches [31, 33] (rkm012_1, mount2), cross-field-based techniques [34,35] (impeller, mid2Fem), interactive methods [36,37] (bunny, CAD4), and octree-based methods [23,29] (anc101, isidore_horse, i29o_bracket, grayloc, bumpy_torus, dancing_children, armadillo, woodenfish, bimba, greek_sculpture). Some of these meshes contain inverted elements, while others have all positive Jacobians but deviated boundaries. We intentionally tangle interior vertices of all meshes (boundary vertices remain unmoved) to increase the difficulty of quality improvement. The same tangled mesh is processed when comparing *HexOpt* with its competitors [21,23,26,28,32]. The performance of *HexOpt* on these 16 models is shown in Fig. 4 and Fig. 5.

As demonstrated in Table 1 and Fig. 7, *HexOpt* produces inversion-free hex meshes H_f while achieving significant improvements in the worst scaled Jacobian. Here, PostSJ denote the minimum scaled Jacobian of the post-optimization state. MaxDist quantifies the bidirectional Hausdorff distance ratio between the boundary $bd(H)$ and the input surface T , normalized by the bounding box diagonal length. This value

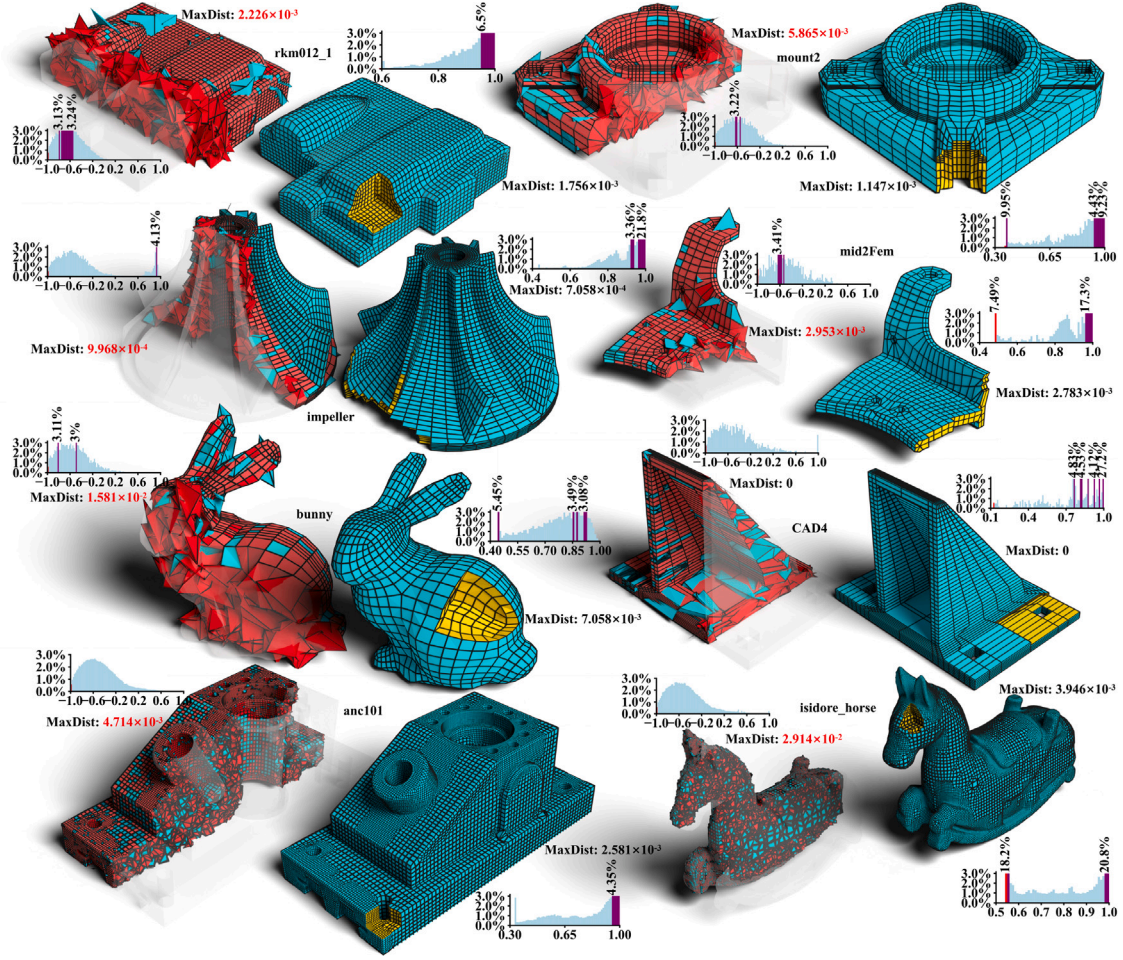


Fig. 4. Optimization results for eight hex models generated by four different methods. The target boundary is displayed translucently. For each model, the left/right figure shows the mesh before/after optimization. Inverted elements are highlighted in red, and non-inverted elements are shown in blue. The mesh interior is shown in yellow with some elements removed. The maximum relative distance and the scaled Jacobian histogram before and after optimization are provided. In the histograms, purple bars are truncated for better visualization due to their high frequency. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

is defined as $\max(d(bd(H), T), d(T, bd(H)))$, where $d(A, B)$ represents the maximum distance from any point on set A to the nearest point on set B . In our implementation, we calculate this by sampling mass vertices on both $bd(H)$ and T . Runtime is reported as L-BFGS/gradient descent for *HexOpt* and as reproduced on the same computer for other methods.

Starting from a perturbed initial mesh (either by giving a Gaussian noise to each internal point or by positioning all internal points to a fixed coordinate), our optimization framework ensures that the minimum scaled Jacobian of the final mesh typically exceeds both the original mesh and prior methods [21,23,26,28,32] while maintaining a slightly smaller MaxDist (i.e., more accurate representation). We compiled and ran [23,26,32] directly, and re-implemented [21,28]. The results show that L-BFGS attains PostSJ values comparable to gradient descent while halving the computation time. Furthermore, the optimization time of *HexOpt* is shorter than that of previous methods for meshes with similar element counts. The only method that is consistently faster than *HexOpt* is [23]. However in [23], the minimum scaled Jacobian is only optimized to a small positive value.

To further validate the robustness of our algorithm, we conduct batch optimization on two of the largest publicly available hex mesh datasets: 4075 octree-based hex meshes from [23] and 111 frame-field-based hex meshes from [42]. The input triangle meshes are extracted

from [43,44], correspondingly. Similarly, to ensure a fair comparison, all competing methods are initialized with the identical input hex mesh for each geometry. Then we ensure that the MaxDist remains similar across all methods for the same geometry. By isolating the MaxDist as a controlled variable, we can directly evaluate and compare PostSJ and Time with each method. Under this constraint, the intersection over union between the volume bounded by the input triangle surface and the volume of the hex mesh after optimization consistently exceeds 0.997, indicating negligible volumetric deviation. Moreover, as shown in Fig. 7, the optimized meshes always maintain a minimum scaled Jacobian greater than zero, and in the vast majority of cases, achieve significantly higher minimum scaled Jacobian values than their pre-optimization counterparts. In Fig. 8, we provide computation time comparison. *HexOpt* is slower than [23], which optimizes the minimum scaled Jacobian only to a small positive threshold, but is faster than all other competing methods.

Additionally, we observe that the method may perform bad with meshes that have large aspect ratio (CAD4). A plausible explanation for this is that the scaled Jacobian of elements with large aspect ratios is highly sensitive to vertex movements along the shorter edges, making it challenging to achieve an optimal solution. From this perspective, the most suitable application for *HexOpt* may be as a post-optimization technique for adaptive meshes. Finally, the convergence plots of the

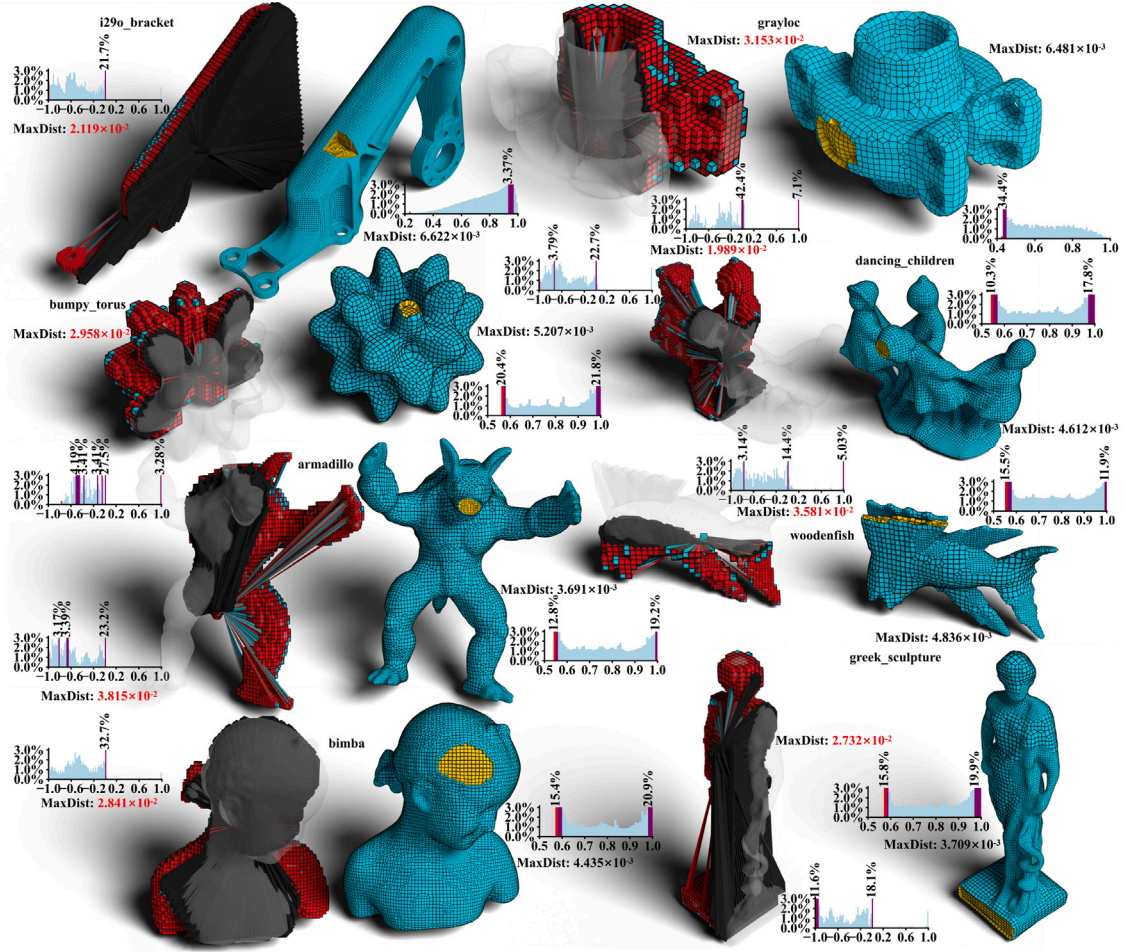


Fig. 5. Optimization results for eight models generated by the octree-based method. The target boundary is displayed translucently. For each model, the left/right figure shows the mesh before/after optimization. Inverted elements are highlighted in red, and non-inverted elements are shown in blue. The mesh interior is shown in yellow with some elements removed. The maximum relative distance and the scaled Jacobian histogram before and after optimization are provided. In the histograms, purple bars are truncated for better visualization due to their high frequency. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Algorithm 1 HexOpt: All-Hexahedral Mesh Quality Improvement

Input: Manifold, watertight triangular mesh T with annotated sharp features, an all-hex mesh H , minimum scaled Jacobian threshold θ

Output: Warped all-hex mesh H_f with improved quality, aiming for better boundary conformity between $bd(H)$ and T .

```

1: Initialize  $N_h \leftarrow \#elem \in H$ ,  $N_{bd} \leftarrow \#vert \in bd(H)$ , iteration number
    $k \leftarrow 0$ , history length  $m \leftarrow 15$ , Lagrangian multiplier  $\lambda \leftarrow 0$ , penalty
   coefficient  $\rho_k \leftarrow 10^{-8}$ , Armijo constant  $c_1 \leftarrow 10^{-4}$ 
2:  $\theta = 0$ 
3: while  $\theta < \theta$  do
4:   while  $\exists v \in bd(H), \|v - v_t\|_2 > 10^{-8}$  do
5:     Call SurfFit to compute  $V_t$   $\triangleright$  Update equality constraints
6:     Call L-BFGS to update variables  $\triangleright$  See Algorithm 3
7:     if  $k \% 100 == 0$  then
8:       LaplacianSmoothing( $H$ )  $\triangleright$  Smooth the mesh
9:     end if
10:     $k \leftarrow k + 1$ 
11:   end while
12:    $\theta \leftarrow \theta + 0.01$ 
13: end while
14: return  $H_f$ 

```

minimum scaled Jacobian for these models are shown in Fig. 6. At each point on the curves, $bd(H_f)$ is exactly fitted to T . From the plots, we observe that it typically takes some time to reach convergence. However, the minimum scaled Jacobian is rapidly improved to 60%–70% of its final value, taking only around 10% of the total time. Subsequently, the rate of improvement slows down. Therefore, in practice, one can stop the optimization process once the convergence curve becomes sufficiently stable, without necessarily waiting for complete convergence.

We investigate the necessity of the ReHQJ objective function configuration by replacing $ReHQJ(h, \theta)$ in Eq. (8) with $ReHJ(h, \theta)$ in Eq. (3). For the i29o_bracket model with a significant scale disparity between the geometric details and the mesh structure, the optimization process cannot reduce MaxDist to below 0.01 while maintaining a positive minimum scaled Jacobian. Furthermore, omitting Jacobian-based optimization of interior vertices and distinct Laplacian smoothing techniques for boundary $bd(H)$ and non-boundary $H \setminus bd(H)$ vertices leads to significantly longer convergence time (more than 10 times) across all eight models in Fig. 5. This divergence arises because $bd(H)$ lies outside the domain T . Moreover, replacing $ReHQJ(h, \theta)$ in Eq. (8) with $ReSJ(h, \theta)$ in Eq. (2) results in unsuccessful convergence for all tested models. To evaluate the necessity of equality constraint configurations in geometry-aware mapping, we first test removing the normal direction guidance module. This significantly degrades optimization

Algorithm 2 *HexOpt*: SurfFit

```

1: Input: Triangle mesh  $T$ , hex boundary vertices  $V^k$ , sharp corners  $V_{\text{corner}}$ , sharp edges  $E_{\text{edge}}$ 
2: Output: Mapped vertices  $V_t^k$ 
3: // Normal-guided vertex projection
4: for each vertex  $v \in V^k$  do
5:   Collect candidate projections  $\{v_t\}$  on all  $t \in T$  via closest-point queries
6:   Determine  $\text{sgn}(v \rightarrow T) \in \{-1, 1\}$  via ray casting  $\triangleright$  -1: inside, 1: outside
7:   for each candidate projection  $v_t$  do
8:     Compute directional metric:
9:      $d_v \leftarrow \sum_{n \in N_v} (1.1 \times \|v_t - v\|_2 + \text{sgn}(v \rightarrow T)(v_t - v) \cdot n)$ 
10:    end for
11:   Update  $v_t \leftarrow \arg \min_{v_t} d_v$ 
12: end for
13: // Bijective mapping for extreme vertices
14: Identify extreme vertex  $v^* \leftarrow \arg \max_v \min_{v_t} \|v - v_t\|_2$ 
15: Project link vertices:  $v_t^i \leftarrow \arg \min_{p \in V(T)} \|v^i - p\|_2, \forall v^i \in Lk(v^*)$ 
16: // Build closed loop on  $T$ :
17: Connect  $\{v_t^1, \dots, v_t^n\}$  preserving  $Lk(v^*)$  order
18: Insert Dijkstra paths between non-adjacent pairs
19: // Parameterize domain
20: Map  $Cl(Sl(v^*))$  to  $P(v^*)$  via mean-value coordinates
21: Transfer barycentric coordinates to  $T$  surface
22: Update  $v_t^*$  position using coordinate transfer
23: // Sharp feature preservation
24: for each sharp corner  $v_{\text{corner}} \in V_{\text{corner}}$  do
25:   Find closest boundary vertex:  $v \leftarrow \arg \min_{u \in bd(H)} \|u - v_{\text{corner}}\|_2$ 
26:   Enforce projection:  $v_t \leftarrow v_{\text{corner}}$ 
27: end for
28: for each sharp edge  $e_{\text{edge}} = (v_{\text{start}}, v_{\text{end}}) \in E_{\text{edge}}$  do
29:   Locate endpoints:  $v_1 \leftarrow \arg \min \|u - v_{\text{start}}\|_2, v_2 \leftarrow \arg \min \|u - v_{\text{end}}\|_2$ 
30:   Compute shortest path  $l \leftarrow (v_1, v^1, \dots, v^n, v_2)$  via Dijkstra
31:   Calculate total length:  $D \leftarrow \sum_{i=1}^{n-1} \|v^{i+1} - v^i\|_2$ 
32:   Initialize cumulative distance:  $d_{\text{acc}} \leftarrow 0$ 
33:   for each path vertex  $v^i \in l$  do
34:      $d_{\text{acc}} \leftarrow d_{\text{acc}} + \|v^i - v^{i-1}\|_2$ 
35:     Compute weight  $\lambda^i \leftarrow d_{\text{acc}}/D$ 
36:     Project to edge  $v_t^i \leftarrow \lambda^i v_{\text{end}} + (1 - \lambda^i) v_{\text{start}}$ 
37:   end for
38: end for
39: return  $V_t^k$ 

```

performance for the bunny and i29o_bracket models. We then test removing the bijective mapping module. While this modification does not result in failure for any model reported in our experiments, it might cause optimization failure for very coarse hex meshes. Finally, we examine the effect of removing the Laplacian smoothing step/replacing it with the naive Laplacian smoothing in Algorithm 1. Although *HexOpt* achieves the same minimum scaled Jacobian and MaxDist values without smoothing, the optimization takes five times longer on average when $bd(H)$ is larger than T . This slowdown occurs because Laplacian smoothing acts as a surface-area minimization process, which naturally contracts $bd(H)$ and accelerates its fitting to T . Additionally, optimized meshes without smoothing exhibit considerably larger edge ratios compared to those with smoothing, as smoothing regularizes inter-vertex distances—though the edge ratio is not considered a mesh quality metric in this paper. When replaced with the naive Laplacian smoothing, there is almost no impact near convex surfaces; however, near concave surfaces, using the naive Laplacian smoothing increases computation

Algorithm 3 *HexOpt*: L-BFGS

```

Input: Boundary vertices  $V^k$ , target boundary vertices  $V_t^k, \rho, \lambda, N_h, N_{bd}, k, m, c_1$ , history vectors  $\rho_i, s_i, y_i, i = k - m, k - m + 1, \dots, k - 1, i \geq 0$ 
Output: Updated boundary vertices  $V^{k+1}$ 
1: Calculate gradient  $q \leftarrow \nabla L_k$   $\triangleright$  See Equation (8)
2: for  $i = k - 1, k - 2, \dots, k - m$  do  $\triangleright$  First loop
3:    $\alpha_i \leftarrow \rho_i s_i^T q$ 
4:    $q \leftarrow q - \alpha_i y_i$ 
5: end for
6:  $r \leftarrow H_k^0 q = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}} q$ 
7: for  $i = k - m, k - m + 1, \dots, k - 1$  do  $\triangleright$  Second loop
8:    $\beta \leftarrow \rho_i y_i^T r$ 
9:    $r \leftarrow r + s_i(\alpha_i - \beta)$ 
10: end for
11:  $V^k \leftarrow (v_0, v_1, \dots, v_{N_{bd}-1})$ 
12:  $V_t^k \leftarrow (v_t^0, v_t^1, \dots, v_t^{N_{bd}-1})$ 
13:  $s_k \leftarrow V^k - V^{k-1}$ 
14:  $y_k \leftarrow \nabla L_k - \nabla L_{k-1}$ 
15: if  $y_k^T s_k == 0$  then
16:    $\rho_k \leftarrow 10^8$ 
17: else
18:    $\rho_k \leftarrow \frac{1}{y_k^T s_k}$ 
19: end if
20:  $\lambda \leftarrow \lambda + \rho(V^k - V_t^k)$   $\triangleright$  Update Lagrange multiplier
21: if  $\sum_{h \in H} ReSJ(h, \theta) = N_h \theta$  then
22:    $\rho \leftarrow 2\rho$   $\triangleright$  Update penalty parameter
23: end if
24: // Armijo Line Search
25:  $a \leftarrow 1$ 
26: while  $a > 10^{-8}$  and  $L(V^k + ar) - L(V^k) > c_1 ar^T \nabla L(V^k)$  do
27:    $a \leftarrow 0.9a$ 
28: end while
29:  $(v_0, v_1, \dots, v_{N_{bd}-1}) \leftarrow V^k + ar$   $\triangleright$  Update variables

```

time. This is because at these locations, Laplacian smoothing sometimes pulls interior points outside the surface, and then Jacobian optimization pulls these points back inside, creating an oscillatory conflict. In contrast, the proposed Laplacian smoothing ensures that interior points are at least not pulled outside the surface, thereby reducing such oscillation. Across all meshes, using the naive Laplacian smoothing takes approximately three times longer on average compared to the proposed Laplacian smoothing.

4. Conclusion and future work

In this paper, we introduce *HexOpt*, a new software package for improving the quality of all-hex meshes. Given a poor-quality or inexact boundary-fitting hex mesh and a triangular geometry (with annotated sharp features) onto which the hex mesh must be projected, *HexOpt* formulates a constrained optimization problem that includes both mesh quality term ReHQJ and geometry-aware fitting constraint with normal direction and local bijective mapping. The algorithm then employs the AL, L-BFGS and Armijo line search methods to minimize the ReHQJ objective function. This approach is robust, efficient, and fully automated, making it particularly suitable for improving mesh quality for complex 3D models. Across all tested models, selected from our group's previous work and what other research archives, our algorithm consistently generates meshes of superior quality compared to the initial state. It is worth noting that the majority of our tests were conducted on octree-based hex meshes, partly due to the higher robustness and wider availability of such datasets compared to alternatives like frame-field-based methods. While *HexOpt* is generally applicable, it has been most

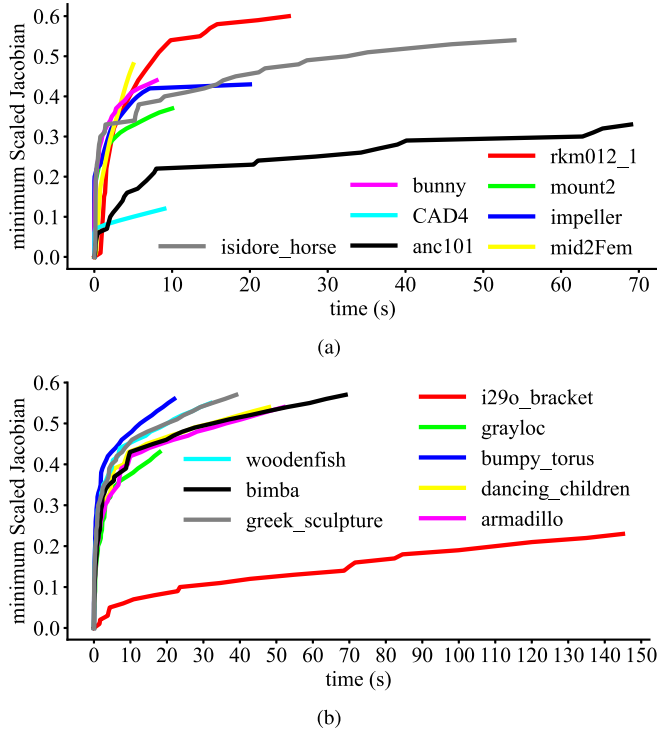


Fig. 6. Convergence plot on the exhibited sixteen models.

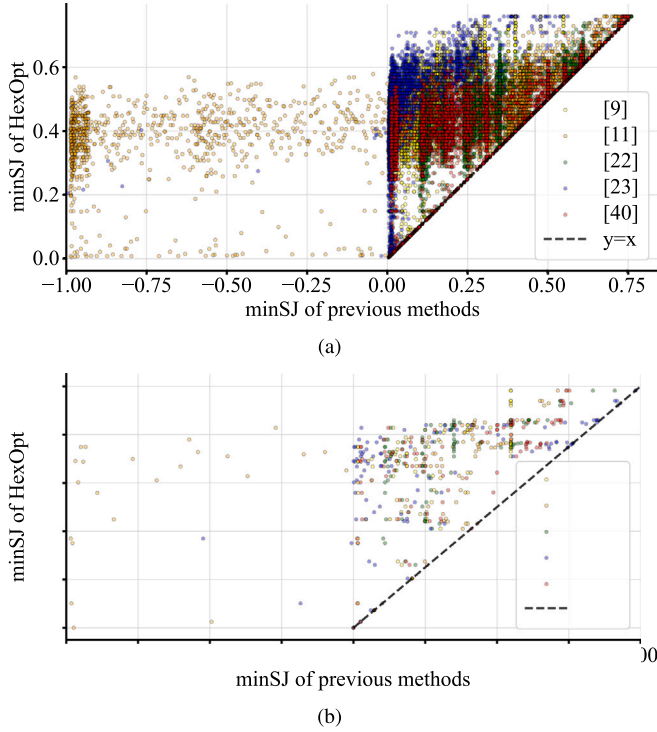


Fig. 7. Minimum scaled Jacobian of (a) 4075 meshes generated by the octree method [23] and (b) 111 meshes generated by the frame-field method [42] after batch optimization by [21,23,26,28,32] (x-axis) and *HexOpt* (y-axis). The results show that *HexOpt* consistently achieves better minimum scaled Jacobian compared to previous methods. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

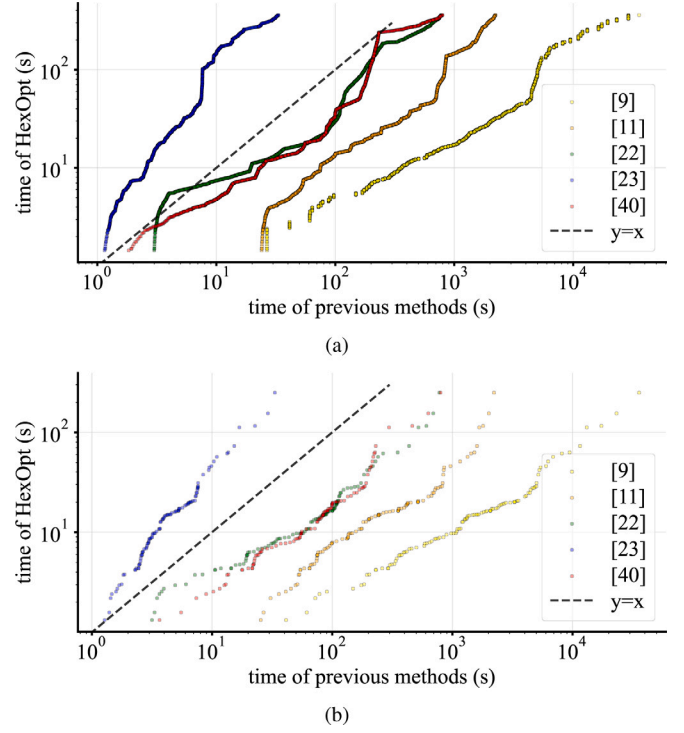


Fig. 8. Computation time of (a) 4075 meshes generated by the octree method [23] and (b) 111 meshes generated by the frame-field method [42] after batch optimization by [21,23,26,28,32] (x-axis) and *HexOpt* (y-axis). The results show that *HexOpt* consistently optimizes mesh quality faster than previous methods, except [23]. However in [23], the minimum scaled Jacobian is only optimized to a small positive value.

thoroughly validated and shows particularly reliable performance in improving octree-generated hex meshes.

While *HexOpt* has demonstrated promising results in achieving rapid, robust, and high-quality all-hex mesh optimization for industrial applications, several challenges remain to advance both theoretical and practical aspects of the framework. First, establishing rigorous theoretical guarantees for optimization performance remains an open challenge. Prior methods, such as vertex-wise optimization for convergence [20] or incremental boundary adjustments to preserve Jacobian positivity [24], ensure monotonic mesh quality improvement but fail to provide a lower bound on quality. These approaches often underperform in practice due to overly restrictive constraints. While quality-guaranteed algorithms have been successfully implemented for quad meshes [45–47] and tetrahedral meshes [48] by leveraging the topological simplicity of 2D Euclidean spaces and 3D simplices, extending such guarantees to all-hex meshes remains a significant hurdle. Future work will prioritize formulating theoretical proofs for mesh quality lower bounds during the generation phase rather than relying on post-processing. Second, the reliability of automatic sharp feature detection is inherently limited in configurations where sharp edges on the input geometry T are densely clustered. Strict alignment of $bd(H)$ to T risks localized Jacobian degradation or failure, and no existing method can preemptively identify such rigid geometric constraints. Third, preserving sharp features becomes infeasible when the required valence at a corner exceeds the maximum allowable valence of adjacent quad boundaries. Beyond these challenges, integrating machine learning techniques – such as those explored in [49] – to minimize reliance on heuristic steps during mesh generation presents a promising avenue for future research.

CRediT authorship contribution statement

Hua Tong: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yongjie Jessica Zhang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

H. Tong and Y. J. Zhang were supported in part by the US National Science Foundation grants CMMI-1953323 and CBET-2332084.

References

- [1] Zhang Y. Geometric modeling and mesh generation from scanned images. Taylor & Francis Group: CRC Press; 2016.
- [2] Benzley SE, Perry E, Merkley K, Clark B, Sjaardama G. A comparison of all hexagonal and all tetrahedral finite element meshes for elastic and elasto-plastic analysis. In: 4th international meshing roundtable. 1995, p. 179–91.
- [3] Shepherd JF, Tuttle CJ, Silva C, Zhang Y. Quality improvement and feature capture in hexahedral meshes. Technical report UUSCI-2006-029, The University of Utah; 2006.
- [4] Owen SJ. A survey of unstructured mesh generation technology. Int Meshing Roundtable 1998;239(267):15.
- [5] Zhang Y. Challenges and advances in image-based geometric modeling and mesh generation. Image-Based Geom Model Mesh Gener 2013;1–10.
- [6] Zhang Y, Hughes TJ, Bajaj CL. An automatic 3D mesh generation method for domains with multiple materials. Comput Methods Appl Mech Engrg 2010;199(5–8):405–15.
- [7] Schneiders R. Algorithms for quadrilateral and hexahedral mesh generation. Proceedings of the VKI lecture series on computational fluid dynamic, 2000.
- [8] Knupp PM. Hexahedral and tetrahedral mesh untangling. Eng Comput 2001;17:261–8.
- [9] Shepherd JF, Johnson CR. Hexahedral mesh generation constraints. Eng Comput 2008;24(3):195–213.
- [10] Pietroni N, Campen M, Sheffer A, Cherchi G, Bommers D, Gao X, Scateni R, Ledoux F, Remacle J, Livesu M. Hex-mesh generation and processing: A survey. ACM Trans Graph 2022;42(2):1–44.
- [11] Tong H, Zhang YJ. MCHex: Marching cubes based adaptive hexahedral mesh generation with guaranteed positive Jacobian. 2025, arXiv:2511.02064.
- [12] Tong H, Zhang YJ. Element-saving hexahedral 3-refinement templates. 2025, arXiv:2512.14862.
- [13] Owen S, Brown N, Chrisochoides N, Garimella R, Gu X, Ledoux F, Lei N, Quadros R, Ray N, Winovich N, et al. A survey of AI methods for geometry preparation and mesh generation in engineering simulation. 2025, arXiv:2512.23719.
- [14] Canann SA, Tristano JR, Staten ML, et al. An approach to combined Laplacian and optimization-based smoothing for triangular, quadrilateral, and quad-dominant meshes. Int Meshing Roundtable 1998;1:479–94.
- [15] Zhang Y, Bajaj C. Adaptive and quality quadrilateral/hexahedral meshing from volumetric data. Comput Methods Appl Mech Engrg 2006;195(9–12):942–60.
- [16] Zhang Y, Bajaj C, Xu G. Surface smoothing and quality improvement of quadrilateral/hexahedral meshes with geometric flow. Commun Numer Methods Eng 2009;25(1):1–18.
- [17] Zhang Y, Bajaj C, Sohn B-S. 3D finite element meshing from imaging data. Comput Methods Appl Mech Engrg 2005;194(48–49):5083–106.
- [18] Freitag LA. On combining Laplacian and optimization-based mesh smoothing techniques. Technical report, Argonne National Lab; 1997.
- [19] Qian J, Zhang Y, Wang W, Lewis AC, Qidwai MS, Geltmacher AB. Quality improvement of non-manifold hexahedral meshes for critical feature determination of microstructure materials. Internat J Numer Methods Engrg 2010;82(11):1406–23.
- [20] Freitag LA, Plassmann P. Local optimization-based simplicial mesh untangling and improvement. Internat J Numer Methods Engrg 2000;49(1–2):109–25.
- [21] Livesu M, Sheffer A, Vining N, Tarini M. Practical hex-mesh optimization via edge-cone rectification. ACM Trans Graph 2015;34(4):1–11.
- [22] Ruiz-Gironés E, Roca X, Sarrate J, Montenegro R, Escobar JM. Simultaneous untangling and smoothing of quadrilateral and hexahedral meshes using an object-oriented framework. Adv Eng Softw 2015;80:12–24.
- [23] Maréchal L. Advances in octree-based all-hexahedral mesh generation: Handling sharp features. In: 18th international meshing roundtable. 2009, p. 65–84.
- [24] Lin H, Jin S, Liao H, Jian Q. Quality guaranteed all-hex mesh generation by a constrained volume iterative fitting algorithm. Comput-Aided Des 2015;67:107–17.
- [25] Qian J, Zhang Y. Automatic unstructured all-hexahedral mesh generation from B-Reps for non-manifold CAD assemblies. Eng Comput 2012;28:345–59.
- [26] Xu K, Gao X, Chen G. Hexahedral mesh quality improvement via edge-angle optimization. Comput Graph 2018;70:17–27.
- [27] Wang R, Zheng Z, Yu W, Shao Y, Gao S. Structure-aware geometric optimization of hexahedral mesh. Comput-Aided Des 2021;138:103050.
- [28] Huang Q, Zhang W-X, Wang Q, Liu L, Fu X-M. Untangling all-hex meshes via adaptive boundary optimization. Graph Model 2022;121:101136.
- [29] Tong H, Halilaj E, Zhang YJ. HybridOctree_Hex: Hybrid octree-based adaptive all-hexahedral mesh generation with Jacobian control. J Comput Sci 2024;78:102278.
- [30] Knupp PM, Ernst C, Thompson DC, Stimpson C, Pebay PP. The verdict geometric quality library. Technical report, Sandia National Laboratories; 2006.
- [31] Guo HX, Liu X, Yan D-M, Liu Y. Cut-enhanced PolyCube-maps for feature-aware all-hex meshing. ACM Trans Graph 2020;39(4). 106–1.
- [32] Gao X, Shen H, Panozzo D. Feature preserving octree-based hexahedral meshing. Comput Graph Forum 2019;38(5):135–49.
- [33] Yu Y, Wei X, Li A, Liu JG, He J, Zhang YJ. HexGen and Hex2Spline: Polycube-based hexahedral mesh generation and spline modeling for isogeometric analysis applications in LS-DYNA. In: Geometric challenges in isogeometric analysis. Springer; 2022, p. 333–63.
- [34] Li Y, Liu Y, Xu W, Wang W, Guo B. All-hex meshing using singularity-restricted field. ACM Trans Graph 2012;31(6):1–11.
- [35] Livesu M, Pietroni N, Puppo E, Sheffer A, Cignoni P. LoopyCuts: Practical feature-preserving block decomposition for strongly hex-dominant meshing. ACM Trans Graph 2020;39(4):121.
- [36] Takayama K. Dual sheet meshing: An interactive approach to robust hexahedralization. Comput Graph Forum 2019;38(2):37–48.
- [37] Zoccheddu F, Gobetti E, Livesu M, Pietroni N, Cherchi G. HexBox: Interactive box modeling of hexahedral meshes. Comput Graph Forum 2023;42(5):e14899.
- [38] Floater MS. Mean value coordinates. Comput Aided Geom Design 2003;20(1):19–27.
- [39] Bertsekas DP. Constrained optimization and Lagrange multiplier methods. Academic Press; 2014.
- [40] Roweis S. Levenberg-Marquardt optimization. Notes, vol. 52, University of Toronto; 1996.
- [41] Bracci M, Tarini M, Pietroni N, Livesu M, Cignoni P. HexaLab. net: An online viewer for hexahedral meshes. Comput-Aided Des 2019;110:24–36.
- [42] Liu H, Bommers D. Locally meshable frame fields. ACM Trans. Graph. 2023;42(4):112.
- [43] Zhou Q, Jacobson A. Thingi10k: A dataset of 10,000 3D-printing models. 2016, arXiv:1605.04797.
- [44] Beaufort P-A, Reberol M, Kalmykov D, Liu H, Ledoux F, Bommers D. Hex me if you can. In: Computer graphics forum. vol. 41, Wiley Online Library; 2022, p. 125–34.
- [45] Liang X, Ebeida MS, Zhang Y. Guaranteed-quality all-quadrilateral mesh generation with feature preservation. Comput Methods Appl Mech Engrg 2010;199(29–32):2072–83.
- [46] Liang X, Zhang Y. Hexagon-based all-quadrilateral mesh generation with guaranteed angle bounds. Comput Methods Appl Mech Engrg 2011;200(23–24):2005–20.
- [47] Liang X, Zhang Y. Matching interior and exterior all-quadrilateral meshes with guaranteed angle bounds. Eng Comput 2012;28(4):375–89.
- [48] Liang X, Zhang Y. An octree-based dual contouring method for triangular and tetrahedral mesh generation with guaranteed angle range. Eng Comput 2014;30:211–22.
- [49] Tong H, Qian K, Halilaj E, Zhang YJ. SRL-assisted AFM: Generating planar unstructured quadrilateral meshes with supervised and reinforcement learning-assisted advancing front method. J Comput Sci 2023;72:102109.